

Project Overview

DEPLOY A SCALABLE GAME SERVER USING EC2, APPLICATION LOAD BALANCER, AUTO SCALING, AND CLOUDWATCH

The Snake Game application is deployed on AWS using a two-tier architecture consisting of a frontend tier and a backend tier. The design keeps the user-facing layer publicly reachable while the application logic and data-handling layer stay isolated from direct internet access, following standard cloud security practice.

The overall goal of the project was to take a simple browser-based game and turn it into a production-style deployment that could survive real traffic, recover from failures on its own, and scale up or down without manual intervention.

Architecture Summary

- Frontend: HTML, CSS, and JavaScript, hosted on an EC2 instance running the Nginx web server. Nginx serves the static game files directly to the browser.
- Backend: Python Flask REST APIs, hosted on a separate EC2 instance placed in a private subnet with no direct public exposure.
- Networking: A custom VPC with public and private subnets, an Internet Gateway, and a NAT Gateway connect and isolate the two tiers.
- Traffic Management: An Application Load Balancer (ALB) sits in front of the backend and distributes requests across healthy instances.
- Elasticity: An Auto Scaling Group (ASG) automatically adds or removes backend instances based on CPU load.
- Access Control: IAM roles are attached to instances instead of hard-coded credentials, and AWS Certificate Manager (ACM) is used for HTTPS.

Deployment Goals

- Keep the frontend publicly accessible while isolating the backend in a private network reachable only through the load balancer.
- Ensure the application can scale automatically with traffic, without downtime during scale-out or scale-in events.

- Apply least-privilege security at every layer — network (VPC/subnets), instance (security groups), and account (IAM policies).
- Make the system self-healing: a failed instance should be detected and replaced automatically.
- Keep the architecture cost-aware by only running the capacity that is actually needed at any given time.

Why a Two-Tier Design

Separating the frontend and backend into two tiers means each layer can be secured, scaled, and updated independently. The frontend tier is optimized for fast static-file delivery, while the backend tier is optimized for compute and API logic. This separation also limits the blast radius of a security issue — even if the frontend were compromised, the backend remains behind its own security group and private subnet boundary.

VPC Configuration

A Virtual Private Cloud (VPC) provides the isolated network environment in which all application resources run. Every other resource in this project — EC2 instances, the load balancer, and the NAT Gateway — lives inside this network boundary.

Setup Steps

1. Create a VPC with CIDR block 10.0.0.0/16, giving roughly 65,000 usable private IP addresses across the network.
2. Create two public subnets across two different Availability Zones (for example 10.0.1.0/24 and 10.0.2.0/24) to host frontend resources and the ALB.
3. Create two private subnets across the same two Availability Zones (for example 10.0.3.0/24 and 10.0.4.0/24) to host backend resources.
4. Attach an Internet Gateway to the VPC to allow inbound and outbound internet access for the public subnets.
5. Create a NAT Gateway inside a public subnet, and allocate an Elastic IP address for it, to provide outbound-only internet access for the private subnets.
6. Create a public route table with a default route (0.0.0.0/0) pointing to the Internet Gateway, and associate it with both public subnets.
7. Create a private route table with a default route (0.0.0.0/0) pointing to the NAT Gateway, and associate it with both private subnets.
8. Verify subnet-to-route-table associations in the VPC console to confirm public subnets can reach the internet directly and private subnets can reach it only outbound.

Why Two Availability Zones

Spreading each subnet pair across two Availability Zones means the loss of a single data center does not take the whole application down. This same multi-AZ layout is reused later by the Application Load Balancer and Auto Scaling Group, so it is worth getting right at this stage.

Security Group Configuration

Security Groups act as virtual, stateful firewalls attached to instances and the load balancer. They permit only the traffic required for the application to function, and every other port stays closed by default.

Frontend Security Group

- Allow HTTP on port 80 from the internet (0.0.0.0/0) so visitors can load the game.
- Allow HTTPS on port 443 from the internet, once a certificate is attached, for encrypted traffic.
- Allow SSH on port 22, restricted to a specific administrator IP address or a bastion host — never opened to 0.0.0.0/0.

Backend Security Group

- Allow Flask application traffic on port 5000, restricted to the ALB security group or the frontend security group only.
- Deny all other inbound traffic, since the backend instance has no public IP and should never be reached directly from the internet.
- Allow outbound traffic so the instance can reach the internet through the NAT Gateway for package installs and updates.

ALB Security Group

- Accept incoming web traffic on port 80 (and 443 if HTTPS is enabled) from the internet.
- Forward validated requests to the backend target group over port 5000, using the backend security group as the only permitted source.

Guiding Principle

Open only the ports required for each tier, and reference security groups by ID rather than by CIDR range wherever possible — this way, if an instance's IP address changes, the rule keeps working automatically because it is tied to the security group, not a specific address

Frontend EC2 Deployment

The frontend is deployed on an Ubuntu EC2 instance in the public subnet, serving static files through Nginx. This tier only needs to serve static assets, so it is intentionally kept simple and lightweight.

Deployment Steps

9. Launch an Ubuntu EC2 instance in a public subnet using a Launch Template, so the configuration is reusable and version-controlled.
10. Attach the frontend security group to the instance so only ports 80, 443, and restricted 22 are reachable.
11. Attach an IAM role to the instance so it has permission to report metrics and logs without storing static credentials.
12. Connect to the instance via SSH and update packages: `sudo apt update && sudo apt upgrade -y`.
13. Install Nginx: `sudo apt install nginx -y`.
14. Upload the HTML, CSS, and JavaScript game files to `/var/www/html/` (or the configured Nginx web root) using SCP or a deployment pipeline.
15. Adjust file and folder permissions so Nginx (running as `www-data`) can read the uploaded files.
16. Start and enable the Nginx service so it survives reboots: `sudo systemctl enable --now nginx`.
17. Test the Nginx configuration for syntax errors: `sudo nginx -t`, then reload if changes were made: `sudo systemctl reload nginx`.
18. Verify the frontend loads correctly in a browser using the instance's public IP address or DNS name.

Automating Future Deployments

Because the frontend Launch Template already installs Nginx and pulls the site files at boot, replacing or scaling this instance later does not require any manual steps — new instances come online already serving the current version of the game.

Backend EC2 Deployment

The backend runs as a Flask REST API on an Ubuntu EC2 instance placed in a private subnet, with no direct route in from the public internet. All traffic must pass through the Application Load Balancer.

Deployment Steps

19. Launch an Ubuntu EC2 instance in a private subnet, using the same Launch Template pattern as the frontend for consistency.
20. Attach the backend security group to the instance so only the ALB (or frontend security group) can reach port 5000.
21. Connect via SSH through a bastion host or AWS Systems Manager Session Manager, since the instance has no public IP address.
22. Install Python, pip, Flask, and Flask-CORS: `sudo apt install python3-pip -y && pip3 install flask flask-cors`.
23. Upload the Flask application code to the instance, either via SCP, a Git pull, or a packaged deployment artifact.
24. Configure environment variables required for secure deployment, such as app secrets, database connection strings, and config flags.
25. Create a systemd service file so the Flask app runs continuously in the background and restarts automatically if it crashes.
26. Implement a health check endpoint at `/api/status` that returns a simple success response, for the Application Load Balancer to monitor.
27. Reload systemd and start the service: `sudo systemctl daemon-reload && sudo systemctl enable --now snake-backend`.
28. Confirm the API responds correctly on port 5000 by curling the health check endpoint locally on the instance..

Application Load Balancer (ALB)

An Internet-facing Application Load Balancer distributes incoming traffic across backend EC2 instances, checks their health, and hides the private backend network from direct exposure.

Setup Steps

29. Create an Internet-facing Application Load Balancer spanning both public subnets, so it survives the loss of one Availability Zone.
30. Create a Target Group for the backend instances, using HTTP as the protocol and port 5000 as the target port.
31. Configure the health check path as `/api/status`, with a reasonable interval and healthy/unhealthy threshold, so only healthy instances receive traffic.
32. Register the backend EC2 instance(s) with the target group, or — once the Auto Scaling Group exists — let it register instances automatically.
33. Create a listener on port 80 that forwards requests to the target group.
34. Optionally attach an AWS Certificate Manager (ACM) certificate to a second listener on port 443 to enable HTTPS end-to-end.
35. Add a redirect rule from HTTP to HTTPS once the certificate is in place, so all traffic is encrypted in transit.
36. Test the setup by sending requests to the ALB's DNS name and confirming they are routed to the backend and answered correctly.

Why the ALB Matters

The load balancer is the single point of entry for backend traffic. It decouples the frontend from any specific backend instance, so instances can be added, removed, or replaced by Auto Scaling without the frontend ever needing to know an IP address changed.

Auto Scaling Group & Scaling Policies

An Auto Scaling Group (ASG) automatically manages the number of EC2 instances to maintain availability and handle changing load, using scaling policies to decide when to add or remove capacity.

Auto Scaling Group Setup

37. Create a Launch Template that defines the AMI, instance type, security group, IAM role, and startup script (user data).
38. Create an Auto Scaling Group using this Launch Template, attached to the private subnets.
39. Set Desired Capacity = 2, Minimum Capacity = 2, and Maximum Capacity = 5, so there is always baseline redundancy.
40. Distribute the Auto Scaling Group across multiple Availability Zones for high availability.
41. Attach the Auto Scaling Group to the ALB target group so new instances register — and are health-checked — automatically.
42. Test resilience by manually terminating one instance and confirming the ASG launches a replacement without manual action.

Scaling Policy Configuration

43. Create a target-tracking (or step) scaling policy based on average CPU utilization across the group.
44. Set a scale-out trigger: launch a new instance when average CPU utilization exceeds 70%.
45. Set a scale-in trigger: terminate an instance when utilization drops back below the defined threshold for a sustained period.
46. Configure a cooldown period to prevent rapid, repeated scaling actions in response to short traffic spikes.
47. Set an instance warm-up time so a new instance is only counted toward capacity once it is fully ready to serve traffic.
48. Optionally configure scheduled or predictive scaling ahead of known traffic patterns, such as a marketing launch.

High Availability and Reliability

The architecture is designed so the application keeps running even if individual components fail, without requiring someone to manually intervene at 2 a.m.

Key Practices

- Deploy frontend and backend instances across multiple Availability Zones, so a single zone outage does not take down the whole application.
 - *Public and private subnets are duplicated in each Availability Zone used.*
- Let the ALB route traffic only to instances that pass their health checks, automatically pulling unhealthy instances out of rotation.
 - *Health checks poll /api/status on a fixed interval with configurable healthy/unhealthy thresholds.*
- Rely on Auto Scaling to detect and automatically replace failed instances, restoring desired capacity without manual action.
- Use health checks at both the ALB level and the application (systemd) level, so a hung process is detected even if the instance itself appears healthy.
- Plan for disaster recovery by documenting how to rebuild the environment from Launch Templates and infrastructure configuration.
- Maintain regular backups of application data and configuration, and periodically test that those backups can actually be restored.

Performance, Cost, and Deployment Summary

Performance Optimization

- Optimize application code paths and database or API calls to reduce response times.
- Enable GZIP compression on Nginx to reduce payload size for static assets and speed up page loads.
- Monitor key performance indicators such as response time and error rate to catch regressions early.

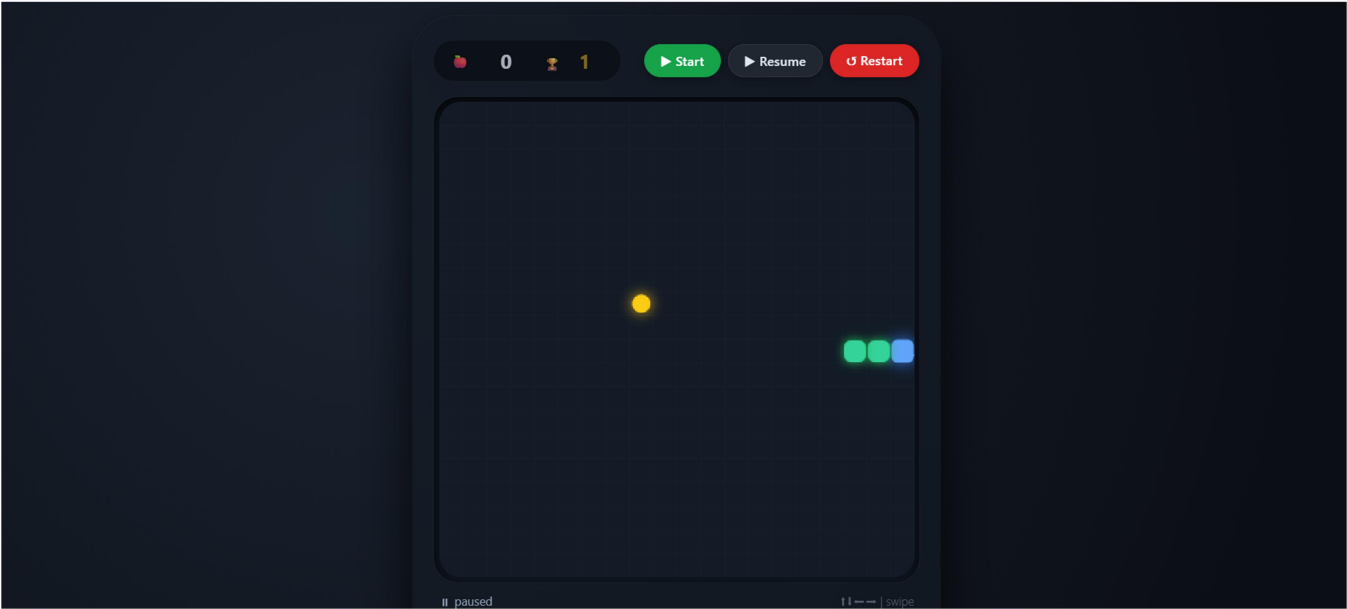
Cost Optimization

- Let Auto Scaling launch additional instances only when actually required, avoiding paying for idle capacity.
- Use AWS Cost Explorer and AWS Budgets to track spend and get alerted before costs run away.
- Consider Reserved or Spot Instances for predictable or flexible workloads to reduce compute cost over time.

End-to-End Deployment Steps (Summary)

49. Create the VPC (10.0.0.0/16) with 2 public and 2 private subnets, an Internet Gateway, and a NAT Gateway.
50. Create the Security Groups: frontend, backend, and ALB, each scoped to only the traffic they need.
51. Launch and configure the frontend EC2 instance with Nginx serving the HTML/CSS/JS game files.
52. Launch and configure the backend EC2 instance with Flask running as a systemd service, exposing /api/status.
53. Create the Application Load Balancer, Target Group, and health check pointing to /api/status.
54. Create the Launch Template and Auto Scaling Group (min 2, desired 2, max 5) across multiple Availability Zones.
55. Attach scaling policies based on CPU utilization, scaling out above 70% utilization.
56. Attach IAM roles to instances and configure ACM for HTTPS on the ALB listener.
57. Verify high availability by testing instance failure and confirming automatic replacement.
58. Review performance and cost settings (GZIP, Cost Explorer, Budgets) before going live.
59. Access the application through the ALB DNS name to confirm the frontend and backend are fully connected end to end.

OUTPUT SCREEN



CLOUD WATCH

Alarms

Mute rules - new

Alarms (4)

Clear selection

Create composite alarm

Actions

Create alarm

Filter alarms

Quick filters

< 1 >

	Name	State	Actions	Actions muted - new	Last state update (UTC)	Condition
☐	TargetTracking-autos-AlarmLow-5ed47f6a-e7a4-429e-9598-9295b58f45d4	⚠ In alarm	✅ Actions enabled	-	2026-07-30 21:07:20	CPUUtil minutes
☐	TargetTracking-autos-AlarmHigh-7372b8ce-e0b0-4b91-b5bc-ff567b1d7b3e	✅ OK	✅ Actions enabled	-	2026-07-30 03:51:06	CPUUtil minutes
☐	TargetTracking-autoscaling-AlarmLow-fcda9c66-1cf4-4537-813c-84a6d2573ab2	⚠ In alarm	✅ Actions enabled	-	2026-07-30 02:30:37	CPUUtil minutes
	TargetTrackinn-					

VS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

EC2 INSTANCE

Instances (5) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	F
☐	front	i-0aa494bac078207ad	Stopped	t2.micro	-	ap-southeast-1c	-	-
☐	back	i-0dbc1ac5000a3f595	Stopped	t2.micro	-	ap-southeast-1c	-	-

LOAD BALANCING

Load balancers (3) What's new?

Actions

Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

< 1 >

☐	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones
☐	Frontend-ALB	Active	application	Internet-facing	IPv4	vpc-0cd7d1bab6b47d2ba	2 Availability Zones
☐	backend-alb	Active	application	Internet-facing	IPv4	vpc-099521fa29e38d62b	2 Availability Zones
☐	back	Active	application	Internet-facing	IPv4	vpc-099521fa29e38d62b	2 Availability Zones

0 load balancers selected

Select a load balancer above.

AUTO SCALING

Auto Scaling groups (3) Info

Last updated less than a minute ago

Launch configurations

Launch templates

Actions

Create Auto Scaling group

Search your Auto Scaling groups

< 1 >

☐	Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired c
☐	autos	At desired capacity	back-g Version Default	1	1/1 Healthy	1
☐	BACKEND	At desired capacity	BACKEND-TM Version Default	1	1/1 Healthy	1
☐	autoscaling	At desired capacity	Frontend-LT Version Default	1	1/1 Healthy	1

VPC

Your VPCs

VPC encryption controls

Your VPCs (3) Info

Last updated less than a minute ago

Actions

Create VPC

Find VPCs by attribute or tag

< 1 >

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 C
☐	frontend	vpc-0cd7d1bab6b47d2ba	Available	-	-	Off	10.0.C
☐	-	vpc-0ca6a85d197939d39	Available	-	-	Off	172.3
☐	backend	vpc-099521fa29e38d62b	Available	-	-	Off	10.0.C